# Oficina
# Tech Challenge - Fase 4

Microsserviços, Saga Pattern e entrega automatizada

**RASCUNHO DE ENTREGA** - identificação dos participantes e/ou vídeo ainda pendentes.

## Participantes

Identificação dos participantes: pendente de preenchimento pelo grupo.

## Repositórios

https://github.com/christochula/oficina-api

https://github.com/christochula/oficina-billing-service

https://github.com/christochula/oficina-production-service

https://github.com/christochula/oficina-auth-serverless

https://github.com/christochula/oficina-infra-kubernetes

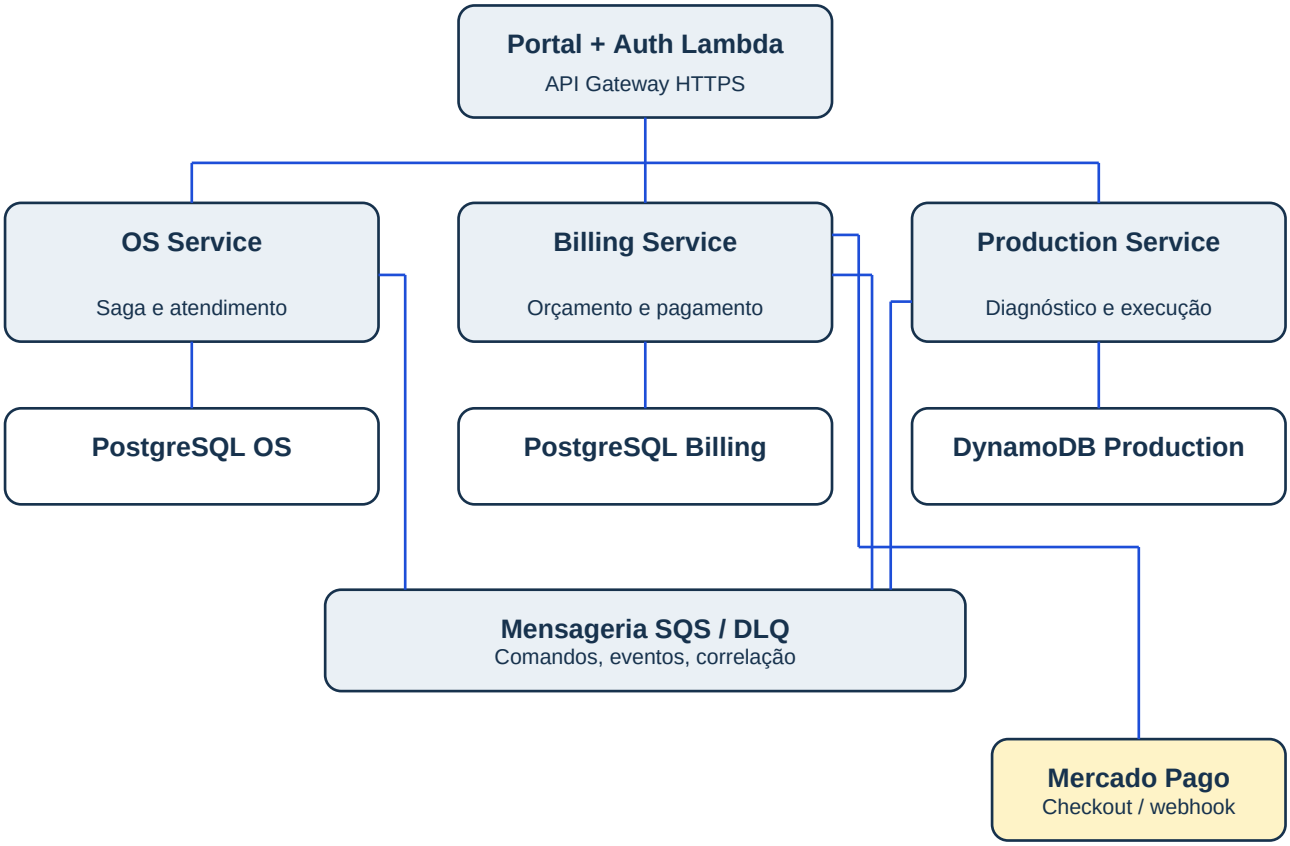
https://github.com/christochula/oficina-infra-database

## Vídeo de demonstração

Pendente de gravação/publicação. Limite: 15 minutos.

# Arquitetura final

Três serviços independentes compartilham o cluster EKS e a observabilidade, mantendo propriedade exclusiva dos dados e infraestrutura de aplicação em seus próprios repositórios.




EKS compartilhado | Bancos exclusivos | Datadog: traces, logs e métricas

| Serviço | Dados e responsabilidade |
|---|---|
| OS / PostgreSQL | Clientes, veículos, usuários, catálogo, OS, histórico e orquestrador persistente. |
| Billing / PostgreSQL | Orçamentos, versões aprovadas, cobrança e confirmação Mercado Pago. |
| Production / DynamoDB | Filas, diagnóstico, peças, reservas, consumo e conclusão técnica. |

# Saga e justificativas

A orquestração reside no OS Service. A decisão centralizada torna as etapas, os prazos técnicos e as compensações visíveis, preservando esperas humanas por aprovação e pagamento.

| Etapa | Avanço / compensação |
|---|---|
| Preparação | Congelar orçamento aprovado, reservar peças, confirmar e liberar execução. |
| Falha antes do reparo | Cancelar reserva/trabalho e liberar orçamento; manter aprovação e histórico. Nova tentativa recebe outro sagaId. |
| Após o reparo | Cobrar e aguardar confirmação. Recusa não desfaz reparo; entrega permanece bloqueada. |
| Pagamento | Validar webhook e consultar recurso no Mercado Pago. Conferir valor, moeda e referência antes de liberar entrega. |

## Tecnologias e consistência

NestJS/TypeScript reduz o esforço de extração da base existente. PostgreSQL atende atendimento e transações financeiras; DynamoDB atende os agregados de produção. SQS desacopla o fluxo com retentativas e DLQ. Outbox/inbox e transações locais evitam perda de intents e duplicação de efeitos.

## Qualidade e demonstração

Os repositórios incluem testes unitários com gate mínimo de 80%, BDD com os três serviços, teste do portal, Docker, Helm e Terraform. O CI executa testes e Sonar antes do deploy. Os READMEs devem apontar para as execuções reais de CI, cobertura e observabilidade.

## Conferências externas antes do envio

Preencher acessos AWS, Sonar, Datadog e Mercado Pago; executar os deploys e o pagamento com contas de teste; registrar os links/evidências; preencher participantes e vídeo no arquivo entrega.json e regenerar este documento.